## G. SOLID Principles

| Principle | How it is applied here |
|---|---|
| Single Responsibility | Booking never prints a ticket — TicketPrinter owns that job. A change to how a ticket looks touches only TicketPrinter; a change to a booking rule touches only Booking. |
| Open/Closed | BookingService only ever talks to the abstract Payment type. Adding NetBanking means adding one new subclass — BookingService's code does not change at all. |
| Liskov Substitution | Any Payment — UpiPayment, CardPayment, CashPayment, or a future NetBankingPayment — can be substituted wherever a Payment is expected. BookingService simply calls pay(total) and trusts every implementation to honour that contract. |
| Interface Segregation | Payment only demands pay() from its subclasses; it deliberately does not force a refund() method on every payment type, since not every payment method can meaningfully support a refund. |
| Dependency Inversion | BookingService receives an already-constructed Payment object from its caller instead of instantiating a CardPayment or UpiPayment itself. The booking logic depends only on the Payment abstraction, never on a concrete payment class. |